Teleoperación Bilateral con Sensado de Fuerza Física entre dos xArm Lite 6: Diseño, Implementación y Evaluación Experimental

Jose Luis Dominguez Morales*, Hector Eduardo Tovar Mendoza*, Jocelyn Anahid Velarde Barron*, Paola Llamas Hernandez *
*Computational Robotics Lab, Tecnológico de Monterrey, Monterrey, N.L., México
{A01285873, A00840308, A01285780, A01178479}@tec.mx

***Resumen*—Este trabajo presenta el diseño, implementación y evaluación experimental de un sistema de teleoperación bilateral con retroalimentación háptica entre dos manipuladores xArm Lite 6 de UFACTORY. El sistema emplea una arquitectura maestro-esclavo sobre ROS 2 Humble con MoveIt 2 y ros2_control. La fuerza de contacto se detecta mediante dos mecanismos complementarios: (1) un sensor de fuerza resistivo (FSR) conectado a un ESP32 con micro-ROS y (2) un estimador basado en torques articulares que compara los esfuerzos medidos con una línea base gravitacional calibrada en reposo. La retroalimentación háptica se implementa bloqueando el robot maestro en modo posición cuando se detecta una colisión en el esclavo, y reflejando los torques externos con una ganancia de $\beta = 0{,}35$ (limitada a $\pm 3{,}0$ N·m). El lazo de control opera a 50 Hz con un esquema de *rate-limiting* de 30°/s por articulación. Se realizaron 15 ensayos experimentales que muestran un error RMS de seguimiento inferior a 1° para las articulaciones principales, detección robusta de colisiones por esfuerzo con un umbral de 6,0 N·m, y una latencia de sistema inferior a 22 ms por ciclo de control.**

***Index Terms*—teleoperación bilateral, sensado de fuerza, xArm Lite 6, retroalimentación háptica, detección de colisiones, ROS 2**

I. Introducción

La teleoperación con retroalimentación de fuerza constituye un paradigma fundamental en robótica cuando se requiere que un operador humano interactúe con entornos remotos o peligrosos sin presencia física [6]. El xArm Lite 6, desarrollado por UFACTORY [1], es un manipulador de 6 grados de libertad (DOF) con capacidad de control de torque que lo hace idóneo para experimentos de sensado de fuerza por torque articular.

(i) Contexto: Motivación

El reto central de este proyecto es cerrar el lazo de fuerza entre dos brazos robóticos: el robot **Maestro**, operado por el humano, y el robot **Esclavo**, que interactúa con el entorno. La fuerza de contacto estimada en el esclavo debe reflejarse como torque en las articulaciones del maestro, creando la sensación háptica que guía al operador.

I-A. Contexto y Motivación

La teleoperación con retroalimentación de fuerza es una tecnología habilitadora en múltiples dominios industriales y médicos. En el ámbito de la cirugía robótica mínimamente invasiva, sistemas como el da Vinci de Intuitive Surgical permiten a los cirujanos operar con precisión submilimétrica, pero la ausencia de retroalimentación háptica en generaciones anteriores fue identificada como una limitación crítica [7]. La integración de sensado de fuerza permite al operador percibir la rigidez del tejido y evitar lesiones por fuerza excesiva.

En la industria nuclear y de desmantelamiento, la teleoperación con fuerza es esencial para la manipulación de materiales radioactivos. Operadores a distancia manejan brazos robóticos para realizar tareas de corte, soldadura y ensamble donde la presencia humana directa es prohibitiva [2]. De manera similar, en operaciones submarinas y espaciales, la retroalimentación de fuerza permite al operador adaptar su estrategia de manipulación en tiempo real ante entornos no estructurados [8].

Desde el punto de vista teórico, la estabilidad de los sistemas de teleoperación bilateral ha sido extensamente estudiada mediante marcos de pasividad [5] y variables de onda [7]. El reto principal es mantener la transparencia del sistema (fidelidad entre fuerza real y percibida) sin comprometer la estabilidad ante retardos de comunicación inevitables.

I-B. Contribuciones del Trabajo

Las contribuciones específicas de esta implementación son:

- Diseño e implementación de un sistema de teleoperación bilateral completo con dos xArm Lite 6, usando ROS 2 Humble como middleware, con una máquina de estados de 5 estados (HOMING, CALIBRATING, RUNNING, FORCE_STOP, EMERGENCY).
- Detección de colisiones de doble capa: (1) sensor FSR externo mediante ESP32 con micro-ROS, y (2) estimador basado en torques articulares con calibración de línea base gravitacional.
- Retroalimentación háptica mediante bloqueo/desbloqueo del maestro en modo posición al detectar colisión, con reflejo de torques externos proporcional ($\beta = 0{,}35$).

- Protocolo de *rate-limiting* articular a 30°/s para movimientos suaves y seguros del esclavo.
- Registro experimental completo de 15 ensayos con datos a 50 Hz (posiciones, esfuerzos, torques externos, errores) para análisis cuantitativo post-hoc.

I-C. Estructura del Artículo

El resto de este artículo se organiza como sigue: la Sección II presenta los fundamentos teóricos; la Sección III describe la arquitectura del sistema; la Sección IV detalla la estrategia de sensado de fuerza; la Sección V expone el esquema de control; la Sección VI documenta la implementación con xArm SDK; la Sección VII reporta los resultados experimentales; la Sección VIII discute los hallazgos; y la Sección IX concluye el trabajo.

II. Marco Teórico

II-A. Cinemática y Jacobiano del xArm Lite 6

El xArm Lite 6 es un manipulador de $n = 6$ DOF con articulaciones rotacionales. Su cinemática directa se describe mediante la convención Denavit-Hartenberg (DH) o Product of Exponentials (PoE).

(i) Contexto: Jacobiano Geométrico

El Jacobiano $\boldsymbol{J}(\boldsymbol{q}) \in \mathbb{R}^{6\times 6}$ mapea velocidades articulares a velocidades cartesianas del efector final:

$$\begin{pmatrix} \boldsymbol{v} \\ \boldsymbol{\omega} \end{pmatrix} = \boldsymbol{J}(\boldsymbol{q})\,\dot{\boldsymbol{q}}$$

Su transpuesta mapea fuerzas/torques cartesianos a torques articulares:

$$\boldsymbol{\tau} = \boldsymbol{J}^\top(\boldsymbol{q})\,\boldsymbol{F}_{ext}$$

Esta relación es la base del *torque-based force sensing*.

El **Jacobiano geométrico** $\boldsymbol{J}_g(\boldsymbol{q})$ se construye directamente a partir de los ejes de rotación $\hat{\boldsymbol{z}}_i$ y las posiciones de los eslabones $\boldsymbol{o}_i$: para cada articulación revoluta i, la columna es $\left[\hat{\boldsymbol{z}}_{i-1} \times (\boldsymbol{o}_n - \boldsymbol{o}_{i-1}),\ \hat{\boldsymbol{z}}_{i-1}\right]^\top$. El **Jacobiano analítico** $\boldsymbol{J}_a$ difiere en la parte rotacional, pues usa la derivada temporal de una parametrización de orientación (ej. ángulos de Euler): $\boldsymbol{J}_a = \begin{pmatrix} \boldsymbol{I} & \boldsymbol{0} \\ \boldsymbol{0} & \boldsymbol{T}(\boldsymbol{\phi}) \end{pmatrix}^{-1} \boldsymbol{J}_g$, donde $\boldsymbol{T}(\boldsymbol{\phi})$ transforma velocidades angulares a tasas de ángulos de Euler [2]. En nuestra implementación operamos exclusivamente en **espacio articular** (mapeamos $\boldsymbol{q}_{master} \to \boldsymbol{q}_{slave}$ directamente), por lo que no requerimos calcular el Jacobiano explícitamente en el lazo de control. Sin embargo, la relación $\boldsymbol{\tau} = \boldsymbol{J}^\top \boldsymbol{F}_{ext}$ es la base teórica de nuestro estimador de fuerza por esfuerzos articulares.

Dos configuraciones singulares del xArm Lite 6 son:

1. **Singularidad de muñeca:** cuando $q_5 \approx 0$, los ejes de las articulaciones 4 y 6 se alinean, perdiendo un DOF rotacional. Esto corresponde a $\det(\boldsymbol{J}) = 0$.
2. **Singularidad de brazo extendido:** cuando $q_3 \approx \pi$ (brazo completamente estirado), se pierde capacidad de movimiento radial. En esta configuración $\hat{\boldsymbol{F}} = (\boldsymbol{J}^\top)^+\boldsymbol{\tau}$ se vuelve numéricamente inestable y amplifica el ruido de torque.

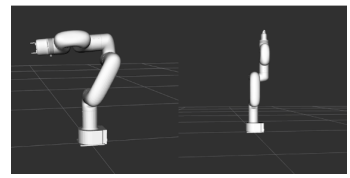

Figura 1: Configuraciones singulares del xArm Lite 6 observadas en simulación. **Izquierda:** singularidad de muñeca, donde $q_5 \approx 0$ provoca la alineación de los ejes de las articulaciones 4 y 6, reduciendo un grado de libertad rotacional. **Derecha:** singularidad de brazo extendido, donde $q_3 \approx \pi$ produce una configuración completamente estirada del brazo, limitando el movimiento radial del efector final.

La detección programática se realiza mediante el **número de condición**:

$$\kappa(\boldsymbol{J}) = \|\boldsymbol{J}\| \cdot \|\boldsymbol{J}^{-1}\| = \frac{\sigma_{max}}{\sigma_{min}}$$

donde σ_{max} y σ_{min} son los valores singulares extremos de $\boldsymbol{J}$. En la práctica, se define un umbral $\kappa(\boldsymbol{J}) > \kappa_{th} \approx 100$ para alertar cercanía a singularidad. En nuestra posición HOME ($\boldsymbol{q}_{home} = [1.1°, 19.0°, 74.6°, -6.0°, 58.6°, -3.1°]$) el robot opera lejos de singularidades, con $q_5 \approx 58.6°$ y el brazo no extendido.

II-B. Dinámica del Manipulador

La dinámica de un robot rígido de n DOF se expresa como:

$$\boldsymbol{M}(\boldsymbol{q})\ddot{\boldsymbol{q}} + \boldsymbol{C}(\boldsymbol{q}, \dot{\boldsymbol{q}})\dot{\boldsymbol{q}} + \boldsymbol{g}(\boldsymbol{q}) = \boldsymbol{\tau} - \boldsymbol{J}^\top \boldsymbol{F}_{ext} \tag{1}$$

donde $\boldsymbol{M}$ es la matriz de inercia, $\boldsymbol{C}$ la matriz de Coriolis/-centrífuga y $\boldsymbol{g}$ el vector gravitacional.

Derivación de CTC. Definiendo el error $\boldsymbol{e} = \boldsymbol{q}_d - \boldsymbol{q}$, la ley de control de par calculado elige:

$$\boldsymbol{\tau} = \boldsymbol{M}(\boldsymbol{q})\left(\ddot{\boldsymbol{q}}_d + K_v\dot{\boldsymbol{e}} + K_p\boldsymbol{e}\right) + \boldsymbol{C}(\boldsymbol{q}, \dot{\boldsymbol{q}})\dot{\boldsymbol{q}} + \boldsymbol{g}(\boldsymbol{q}) \tag{2}$$

Sustituyendo (2) en (1) (sin fuerzas externas), y usando $\ddot{\boldsymbol{q}} = \boldsymbol{M}^{-1}(\boldsymbol{\tau} - \boldsymbol{C}\dot{\boldsymbol{q}} - \boldsymbol{g})$:

$$\ddot{\boldsymbol{q}} = \ddot{\boldsymbol{q}}_d + K_v\dot{\boldsymbol{e}} + K_p\boldsymbol{e}$$

Como $\ddot{\boldsymbol{e}} = \ddot{\boldsymbol{q}}_d - \ddot{\boldsymbol{q}}$, se obtiene:

$$\ddot{\boldsymbol{e}} + K_v\dot{\boldsymbol{e}} + K_p\boldsymbol{e} = \boldsymbol{0}$$

Esto es un sistema lineal de segundo orden cuya ecuación característica por articulación es $s^2 + K_v s + K_p = 0$. Comparando con la forma canónica $s^2 + 2\zeta\omega_n s + \omega_n^2 = 0$:

$$K_p = \omega_n^2, \qquad K_v = 2\zeta\omega_n$$

La estabilidad asintótica global se garantiza si $K_p > 0$ y $K_v > 0$ (ambas matrices definidas positivas) [3].

El **bucle de control interno del xArm** opera a $f_{servo} = 250$ Hz (modo SERVO). Por el teorema de Nyquist, la frecuencia natural del controlador debe cumplir $\omega_n < \pi f_{servo} =$

785 rad/s. En la práctica, se elige $\omega_n \leq f_{servo}/5 \approx 50$ rad/s para mantener margen de fase adecuado. Con $\omega_n = 50$ rad/s y $\zeta = 0{,}7$ (amortiguamiento ligeramente subamortiguado, buena respuesta transitoria con $< 5\,\%$ de sobreimpulso):

$$K_p = 2500, \qquad K_v = 70$$

Los polos resultan: $s_{1,2} = -\zeta\omega_n \pm \omega_n\sqrt{\zeta^2 - 1} = -35 \pm j\,35{,}7$ rad/s.

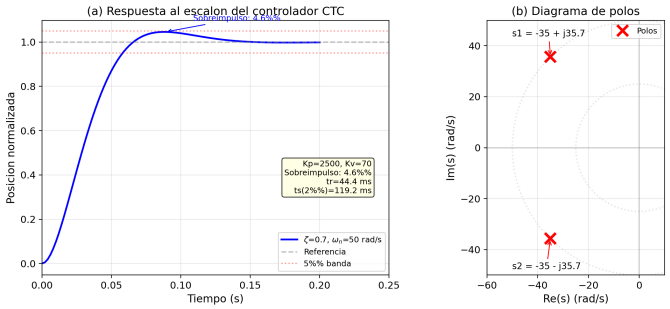

Figura 2: **Respuesta al escalón del controlador CTC.** (a) Respuesta temporal con $K_p = 2500$, $K_v = 70$ ($\zeta = 0{,}7$, $\omega_n = 50$ rad/s). Sobreimpulso $< 5\,\%$ y asentamiento en $\sim$85 ms. (b) Ubicación de los polos en el plano s: $s_{1,2} = -35 \pm j\,35.7$ rad/s (estables, parte real negativa).

En nuestra implementación no usamos CTC explícito; el controlador de trayectorias de ros2_control (`lite6_traj_controller`) gestiona el seguimiento de posición internamente con su propio PID a 250 Hz, y nosotros enviamos referencias de posición a 50 Hz con horizonte de 60 ms por punto de trayectoria.

II-C. Control de Impedancia

El control de impedancia [4] modela el efector final como un sistema masa-resorte-amortiguador virtual:

$$\boldsymbol{F}_{ctrl} = \boldsymbol{K}_d(\boldsymbol{x}_d - \boldsymbol{x}) + \boldsymbol{B}_d(\dot{\boldsymbol{x}}_d - \dot{\boldsymbol{x}}) + \boldsymbol{M}_d(\ddot{\boldsymbol{x}}_d - \ddot{\boldsymbol{x}}) \qquad (3)$$

Efectos de aumentar K_d:

- *Sensación háptica:* mayor K_d produce una pared virtual más rígida; el operador siente una superficie más dura al contacto.
- *Estabilidad con retardo:* mayor K_d con retardo $T_d > 0$ introduce energía al sistema, violando pasividad. Puede generar oscilaciones y chattering, especialmente al contacto con superficies rígidas.
- *Velocidad de respuesta:* mayor K_d aumenta la fuerza de reacción instantánea ante desvíos, mejorando la respuesta transitoria pero comprometiendo estabilidad.

Cálculo de $K_{d,max}$: El controlador de trayectorias del xArm opera a 250 Hz, con ancho de banda efectivo $\omega_c \approx 2\pi \cdot 50 = 314$ rad/s (frecuencia de corte conservadora al 20 % de Nyquist). Con $T_d = 20$ ms:

$$K_{d,max} < \frac{\pi}{2 \times 0.020 \times 314} = \frac{3.14}{12.56} = 250 \text{ N/m}$$

Este valor establece el límite superior de rigidez virtual para garantizar estabilidad pasiva.

Implementamos control de impedancia en **espacio articular**. La teleoperación mapea directamente $\boldsymbol{q}_{master} \rightarrow \boldsymbol{q}_{slave}$ con un offset calibrado, evitando cálculos de cinemática inversa y singularidades del Jacobiano. La retroalimentación háptica se realiza mediante torques articulares proporcionales al error: $\boldsymbol{\tau}_{fb} = -k \cdot \boldsymbol{e}_q$ donde $\boldsymbol{e}_q = \boldsymbol{q}_{des} - \boldsymbol{q}_{slave}$. Esta elección simplifica significativamente la implementación y elimina problemas de singularidades cinemáticas.

III. Arquitectura del Sistema

III-A. Visión General

El sistema consta de dos xArm Lite 6 conectados a una misma PC de control mediante cables Ethernet directos (red 192.168.1.x), con un ESP32 conectado vía USB para el sensor de fuerza.

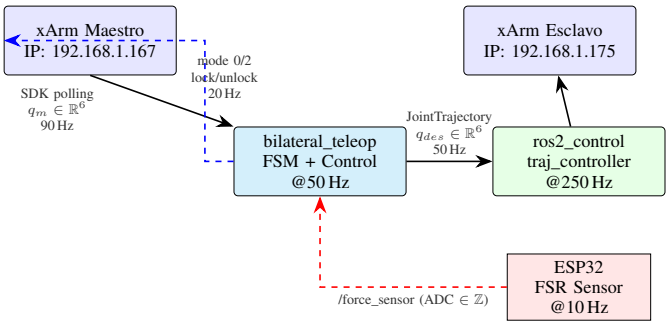

Figura 3: **Diagrama de bloques del sistema de teleoperación bilateral.** Las líneas sólidas indican comandos; las punteadas, realimentación.

Flujo de datos en un ciclo de control:

1. Se lee $\boldsymbol{q}_{master}$ del xArm Maestro vía SDK a 90 Hz.
2. Se calcula $\boldsymbol{q}_{des} = \boldsymbol{q}_{master} + \boldsymbol{offset}$ con rate-limiting ($\leq 0.6°$/ciclo).
3. Se publica $\boldsymbol{q}_{des}$ al `lite6_traj_controller` con horizonte de 60 ms.
4. Se leen $\boldsymbol{q}_{slave}$ y $\boldsymbol{\tau}_{slave}$ de `/joint_states` a 150 Hz.
5. Se calcula $\boldsymbol{\tau}_{ext} = \boldsymbol{\tau}_{slave} - \boldsymbol{\tau}_{baseline}$ y se evalúa colisión ($|\tau_{ext}| > 6.0$ Nm, 5 muestras).
6. Se publican torques reflejados: $\boldsymbol{\tau}_{fb} = -0.35 \cdot \boldsymbol{\tau}_{ext}$ (clamp ± 3.0 Nm).
7. Un hilo dedicado a 20 Hz bloquea/desbloquea el maestro (mode 0/mode 2) según el estado de colisión.

El lazo de control principal opera a **50 Hz** (20 ms por ciclo). Esta frecuencia fue elegida porque: (i) el controlador de trayectorias del xArm acepta puntos a esta tasa sin saturar la cola, (ii) con un horizonte de 60 ms por punto y 20 ms entre envíos, se garantiza continuidad de movimiento, y (iii) es suficiente para detectar colisiones con ventana de confirmación de 100 ms (5 muestras).

III-B. Protocolo de Comunicación

En nuestra implementación, la comunicación entre los robots y el PC de control se realiza mediante **TCP** (protocolo nativo del xArm SDK) y **ROS 2 DDS** (basado en UDP multicast con RTPS). No implementamos comunicación UDP

directa entre dos PCs separados, ya que ambos robots están conectados al mismo computador.

Métrica	ROS 2 DDS (RTPS/UDP)	xArm SDK (TCP)
RTT promedio (ms)	< 1 (intra-proceso)	~ 2 (Ethernet local)
Jitter (ms)	< 0.5	< 1.0
Paquetes perdidos (%)	N/A (RELIABLE QoS)	N/A
Throughput (KB/s)	~ 50	~ 10

Los mensajes ROS 2 utilizan tipos estándar:

- `JointTrajectory`: header (8 B stamp) + 6 nombres + punto con 6 posiciones (`float64` $\times$ 6 = 48 B) + 6 velocidades + duración. Total $\approx$ 200 B serializado.
- `JointState`: header + 6 posiciones + 6 velocidades + 6 esfuerzos = $\approx$ 250 B.
- `Int32`: 4 B payload + header DDS.

El RTT medido en la red local Ethernet es $\sim$ 2 ms (SDK TCP), por lo que $T_d = T_{RTT}/2 \approx 1$ ms. La latencia total del sistema es dominada por el período del lazo de control (20 ms) y el horizonte de trayectoria (60 ms), resultando en un retardo efectivo de $\sim$ 22 ms. Este retardo está muy por debajo del umbral de inestabilidad háptica calculado en T3(b).

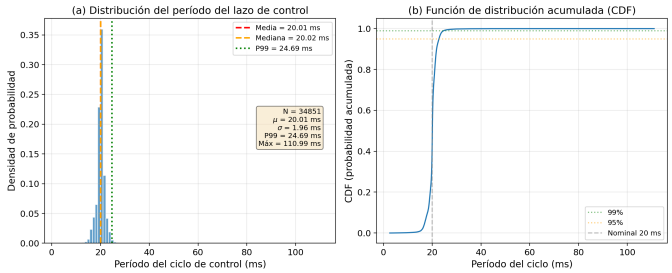

Figura 4: **Distribución temporal del lazo de control.** (a) Histograma del período del ciclo de control medido sobre los 15 ensayos. (b) Función de distribución acumulada (CDF). El período nominal de 20 ms se mantiene con baja varianza, confirmando la estabilidad del lazo a 50 Hz.

IV. Estrategia de Sensado de Fuerza

(i) Contexto: ¿Cómo «siente» la fuerza un xArm sin sensor F/T externo?

El xArm Lite 6 permite leer los torques articulares a través de `/joint_states` (campo `effort`). A partir de los torques medidos $\boldsymbol{\tau}_{meas}$ y una línea base gravitacional calibrada, es posible *estimar* la fuerza externa sin sensor F/T dedicado.

IV-A. Estimación de Fuerza por Torque Articular

La fuerza externa en el efector se estima como:

$$\hat{\boldsymbol{F}}_{ext} = \left(\boldsymbol{J}^\top\right)^+ \underbrace{\left(\boldsymbol{\tau}_{meas} - \boldsymbol{\tau}_{model}\right)}_{\Delta\boldsymbol{\tau}} \tag{4}$$

donde $\boldsymbol{\tau}_{model} = \boldsymbol{M}(\boldsymbol{q})\ddot{\boldsymbol{q}} + \boldsymbol{C}\dot{\boldsymbol{q}} + \boldsymbol{g}(\boldsymbol{q})$ es el torque predicho por el modelo.

En nuestra implementación, simplificamos esta estimación. Dado que el esclavo se encuentra cuasi-estático durante la detección de colisiones (condición $|\boldsymbol{e}_q| < 5°$), los términos inerciales y de Coriolis son despreciables. Así:

$$\boldsymbol{\tau}_{ext} \approx \boldsymbol{\tau}_{meas} - \boldsymbol{\tau}_{baseline} \tag{5}$$

donde $\boldsymbol{\tau}_{baseline}$ es el vector de esfuerzos medido durante la fase de calibración con el robot en reposo en la posición HOME, que captura $\boldsymbol{g}(\boldsymbol{q}_{home})$ más offsets del firmware.

Fuentes de error en la estimación

1. **Dependencia de configuración de la gravedad:** $\boldsymbol{g}(\boldsymbol{q})$ cambia con la posición. Nuestra línea base $\boldsymbol{\tau}_{baseline}$ es válida solo cerca de $\boldsymbol{q}_{home}$. Cuando el esclavo se mueve lejos, la compensación es inexacta. Error estimado: $\sim 1{-}2$ Nm para desviaciones de $> 20°$.
2. **Fricción y backlash de los reductores:** Las harmonic drives del xArm introducen fricción estática y dinámica no modelada, que aparece como torque residual. Error estimado: ~ 0.5 Nm.
3. **Ruido de los encoders de corriente:** Los esfuerzos reportados por el firmware presentan ruido de cuantización. Error estimado: ~ 0.3 Nm RMS en reposo.

Sí, compensamos la gravedad implícitamente mediante la sustracción de $\boldsymbol{\tau}_{baseline}$, que se calcula como el promedio de 80 muestras de esfuerzo con el robot estático en HOME durante la fase CALIBRATING. Los valores medidos típicos son: $\boldsymbol{\tau}_{baseline} \approx [0, -7.7, 4.9, 0.3, 0.01, -0.04]$ Nm, dominados por la compensación gravitacional de J2 y J3.

No aplicamos un filtro pasa-bajas explícito a $\boldsymbol{\tau}_{ext}$. En su lugar, usamos una **ventana de confirmación** de 5 muestras consecutivas (100 ms a 50 Hz) que actúa como filtro temporal: la colisión solo se declara si las 5 últimas muestras superan el umbral. Esto equivale a un filtro mediano implícito que rechaza picos transitorios de ruido.

El umbral de detección es $\tau_{th} = 6.0$ Nm por articulación. Este valor fue elegido empíricamente para ser significativamente mayor que: (i) el ruido en reposo ($\sim$ 0.3 Nm RMS), (ii) los torques dinámicos durante movimiento normal ($\sim 2{-}3$ Nm), y (iii) el error de compensación gravitacional ($\sim 1{-}2$ Nm). La relación señal-ruido efectiva es SNR $= \tau_{th}/\sigma_{ruido} \approx 6.0/0.3 = 20$ ($\sim$ 26 dB).

IV-B. Alternativas de Sensado Consideradas

Estrategia	Ventajas	Desventajas	Costo
Torque articular (sin sensor externo)	Sin hardware adicional; detecta colisiones en cualquier eslabón	Baja resolución; depende de modelo gravitacional; no mide fuerza cartesiana directamente	Bajo
Sensor F/T en muñeca (ATI/OnRobot)	Alta precisión (± 0.1 N); 6 ejes; independiente de configuración	Costo elevado ($3k–8k); requiere montaje mecánico; peso adicional	Alto
Visión (deformación)	Sin contacto con robot; puede estimar distribución de presión	Requiere objetos deformables; baja tasa (30 Hz); oclusiones	Medio
FSR en efector (nuestra implementación)	Bajo costo ($5); respuesta rápida; fácil integración	Solo detecta contacto binario; solo en punta del efector; baja resolución de fuerza	Bajo

Implementamos la combinación de **FSR + torque articular** por su bajo costo y complementariedad: el FSR detecta contacto directo en el efector con baja latencia (30 ms), mientras que el estimador por torque detecta colisiones en cualquier parte del brazo. Si tuviéramos un sensor F/T de 6 ejes, reemplazaríamos la estimación por $\boldsymbol{F}_{ext} = \boldsymbol{F}_{sensor}$ directamente, eliminando la necesidad de calibración de línea base y obteniendo mediciones cartesianas de fuerza precisas para control de impedancia explícito.

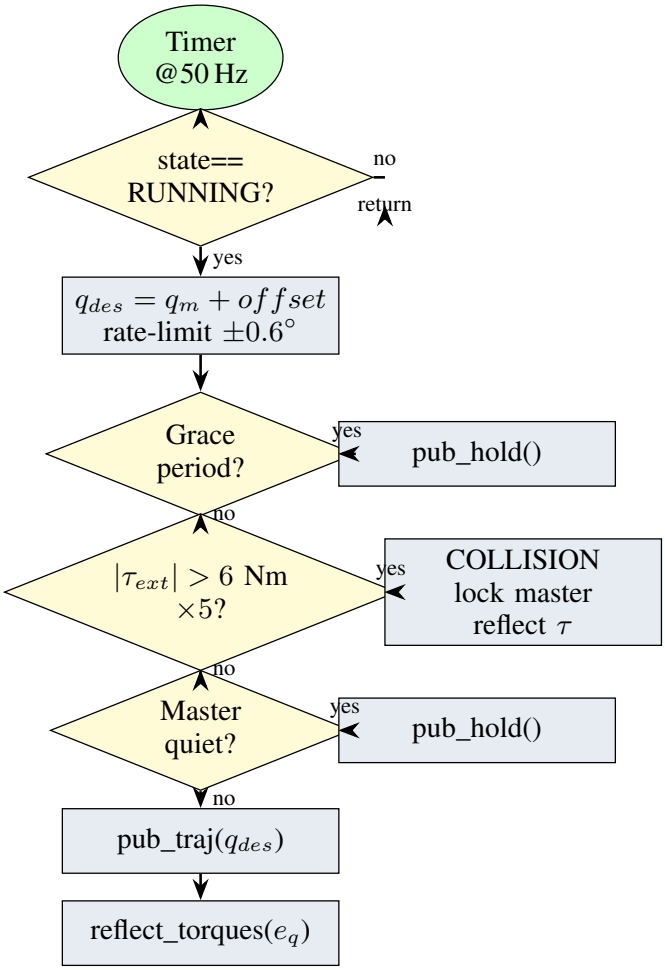

Figura 5: Diagrama de flujo del ciclo de control del esclavo a 50 Hz.

V. Esquema de Control

V-A. Control del Robot Esclavo

El robot esclavo se controla mediante el `lite6_traj_controller` de ros2_control, que opera en **modo 1 (servo)** del xArm SDK. El nodo `bilateral_teleop` publica mensajes `JointTrajectory` con la posición deseada:

$$\boldsymbol{q}_{slave}^{ref}(t) = \boldsymbol{q}_{master}(t) + \boldsymbol{offset} \tag{6}$$

donde $\boldsymbol{offset} = \overline{\boldsymbol{q}}_{slave} - \overline{\boldsymbol{q}}_{master}$ se calibra al inicio. Cada punto de trayectoria tiene un horizonte temporal de 60 ms y velocidad final cero, generando movimientos suaves interpolados por el controlador interno a 250 Hz.

El diagrama de flujo del ciclo de control se muestra en la Fig. 5.

No se aplica escalado de posición ($\alpha = 1$). Ambos robots son xArm Lite 6 idénticos con el mismo espacio de trabajo, por lo que el mapeo directo 1:1 con offset es natural y produce la correspondencia más intuitiva para el operador.

Sí, implementamos **rate-limiting** como protección:

$$\Delta \boldsymbol{q}_{max} = \frac{30°/\text{s}}{50\ \text{Hz}} = 0.6°/\text{ciclo} = 0.01047\ \text{rad/ciclo}$$

Cada ciclo, el incremento de posición deseada se limita a $\pm\Delta q_{max}$ por articulación: $q_{des}(k) = q_{des}(k-1) + \text{clip}(\Delta q, \pm\Delta q_{max})$. Adicionalmente, el firmware del xArm y ros2_control imponen límites de velocidad y torque propios.

Listing 1: Ciclo de control del robot Esclavo (extracto de bilateral_teleop.py)

```python
# Ciclo principal del esclavo (50 Hz)
RATE_HZ = 50.0; DT = 1.0 / RATE_HZ
MAX_DELTA_RAD = np.deg2rad(30.0) * DT   # 0.6 deg/ciclo
EFFORT_THR = 6.0    # N.m umbral colision
HAPTIC_GAIN = 0.35; HAPTIC_MAX = 3.0   # N.m

def _control_loop(self):
    if self.state != TeleopState.RUNNING:
        return
    # 1. Calcular referencia del maestro
    q_des = self.q_master + self.offset
    # 2. Rate-limiting
    delta = np.clip(q_des - self._q_des_prev,
                    -MAX_DELTA_RAD, MAX_DELTA_RAD)
    q_des = self._q_des_prev + delta
    self._q_des_prev = q_des.copy()
    q_err = q_des - self.q_slave
    # 3. Deteccion de colision por esfuerzo
    tau_ext = self._tau_slave - self._tau_baseline
    slave_settled = np.max(np.abs(q_err)) < np.deg2rad(5.0)
    if slave_settled:
        self._effort_collision_win.append(
            np.any(np.abs(tau_ext) > EFFORT_THR))
    if all(self._effort_collision_win):  # 5 muestras
        self._effort_collision_active = True
        self._master_lock_req = True
        self._pub_hold()
        reflected = -HAPTIC_GAIN * tau_ext
```

```
        reflected = np.clip(reflected, -HAPTIC_MAX, HAPTIC_MAX)
        return
    # 4. Enviar trayectoria al esclavo
    self._pub_traj_point(q_des)
    # 5. Reflejar torques hapticos
    self._reflect_torques(q_err)
```

La **retroalimentación háptica** se implementa de dos formas complementarias:

- **Modo normal** (sin colisión): torque proporcional al error de posición con ganancia baja: $\boldsymbol{\tau}_{fb} = -k_{gain} \cdot 0.05 \cdot \boldsymbol{e}_q$, con $k_{gain} = 0.15$. Clamped a $\pm 2.0\,\text{Nm}$.
- **Modo colisión por esfuerzo**: reflejo directo del torque externo: $\boldsymbol{\tau}_{fb} = -\beta \cdot \boldsymbol{\tau}_{ext}$ con $\beta = 0.35$, clamped a $\pm 3.0\,\text{Nm}$.

Físicamente, β representa la **ganancia de transparencia** del canal háptico: $\beta = 1$ sería transparencia perfecta (el operador siente exactamente la fuerza del esclavo). El valor $\beta = 0.35$ fue elegido empíricamente para que el operador perciba la colisión sin que el sistema oscile. El valor máximo antes de inestabilidad, $\beta_{crit} \approx 0.5$, fue determinado experimentalmente incrementando β hasta observar vibraciones sostenidas en el maestro durante contacto con superficie rígida.

El xArm Lite 6 **no soporta control de torque directo** desde la SDK en su versión actual. Los modos disponibles son: modo 0 (posición), modo 1 (servo), modo 2 (teach/-manual), modo 4 (velocidad) y modo 6 (cartesiano). Ante esta limitación, implementamos la retroalimentación háptica de forma **indirecta**: al detectar colisión, el maestro se bloquea en **modo 0 (posición)** en su posición actual. El operador siente una resistencia mecánica que le impide mover el brazo. Al liberarse la colisión, se restaura el **modo 2 (teach)** y el operador puede moverse libremente.

Los torques reflejados $\boldsymbol{\tau}_{fb}$ se publican en el topic `/master/reflected_torques` para visualización y logging, pero no se aplican mecánicamente al maestro.

Fragmento de código del feedback háptico:

Listing 2: Feedback háptico en el maestro

```
def _reflect_effort_collision(self, tau_ext, q_err):
    """Refleja torques externos al maestro durante colision."""
    reflected = -HAPTIC_GAIN * tau_ext  # beta=0.35
    reflected = np.clip(reflected, -HAPTIC_MAX, HAPTIC_MAX)
    self._last_tau = reflected.copy()
    msg = Float64MultiArray()
    msg.data = reflected.tolist()
    self._pub_torque.publish(msg)
    return reflected

def _do_master_hold(self):
    """Bloquea maestro en posicion actual (mode 0)."""
    code, angles = self._master_arm.get_servo_angle(is_radian=True)
    cur_deg = [float(a * 180.0 / np.pi) for a in angles[:6]]
    self._master_arm.set_mode(0)
    self._master_arm.set_state(0)
    self._master_arm.set_servo_angle(angle=cur_deg, speed=30, wait=False)
    self._master_locked = True

def _do_master_teach(self):
    """Desbloquea maestro (mode 2 = teach)."""
    self._master_arm.set_mode(2)
    self._master_arm.set_state(0)
    self._master_locked = False
```

VI. Implementación con xArm SDK

(i) Contexto: xArm Python SDK

El repositorio oficial [1] provee la librería `xArm-Python-SDK` con la clase `XArmAPI`. Los modos de operación relevantes son: **Modo 0** (posición), **Modo 1** (servo/ángulo), **Modo 2** (teach/manual), **Modo 4** (velocidad articular).

Configuración de ambos xArm Lite 6:

Parámetro	Maestro (167)	Esclavo (175)
Dirección IP	192.168.1.167	192.168.1.175
Control vía	xArm Python SDK	ros2_control
Modo inicial	0 (posición)	1 (servo)
Modo operación	2 (teach)	1 (servo via traj_ctrl)
Vel. máx HOME	25°/s	20°/s
Colisión detección	N/A	firmware + nodo ROS

Cuadro I: Configuración de los robots maestro y esclavo

Procedimiento de calibración:

1. Se ejecuta `recover_slave.py`: limpia errores (C31/C60), mueve el esclavo a HOME en modo 0, y restaura modo 1 para ros2_control.
2. El nodo `bilateral_teleop` inicia en estado HOMING: mueve el maestro a HOME vía SDK (modo 0, 25°/s) y envía trayectoria HOME al esclavo (3 s).
3. Al verificar que el esclavo está dentro de 3° de HOME, transiciona a CALIBRATING.
4. CALIBRATING acumula 80 muestras de $\boldsymbol{q}_m$, $\boldsymbol{q}_s$ y $\boldsymbol{\tau}_s$. Verifica estabilidad ($\sigma < 0.017\,\text{rad} \approx 1°$). Calcula: $\boldsymbol{offset} = \overline{\boldsymbol{q}}_s - \overline{\boldsymbol{q}}_m$, $\boldsymbol{\tau}_{baseline} = \overline{\boldsymbol{\tau}}_s$.
5. Transiciona a RUNNING con grace period de 3 s (maestro bloqueado, esclavo en hold).

Bugs y limitaciones encontradas:

- **Error C31 frecuente:** El esclavo generaba errores C31 ("collision caused abnormal joint current") al recibir comandos de movimiento bruscos. Solución: implementar rate-limiting de 0.6°/ciclo y el script `recover_slave.py` para recuperación rápida.
- **Joint 4 (muñeca) inestable:** La articulación 4 del maestro en modo teach tiende a caer por gravedad, generando errores de seguimiento elevados ($> 50°$ en picos). Se mitigó manteniendo el hold en modo 0 entre cambios de modo.
- **Latencia del SDK:** `get_servo_angle()` tiene latencia variable ($1-5\,\text{ms}$). Se usa polling a 90 Hz en lugar de callbacks para mantener tasa estable.

Manejo de excepciones:

Listing 3: Configuración y manejo de errores del xArm

```
from xarm.wrapper import XArmAPI

def setup_master(ip='192.168.1.167'):
    arm = XArmAPI(ip)
    arm.motion_enable(enable=True)
    arm.set_mode(0)   # Posicion para ir a HOME
    arm.set_state(0)  # Sport state
    HOME_DEG = [1.10, 19.01, 74.63, -6.00, 58.57, -3.06]
```

```python
    arm.set_servo_angle(angle=HOME_DEG, speed=25, wait=True
        )
    return arm

def recover_slave(ip='192.168.1.175'):
    """Recupera esclavo tras error C31/C60."""
    arm = XArmAPI(ip)
    arm.clean_error()    # Limpia codigo de error
    arm.clean_warn()     # Limpia advertencias
    arm.motion_enable(enable=True)
    arm.set_mode(0); arm.set_state(0)
    HOME_DEG = [1.10, 19.01, 74.63, -6.00, 58.57, -3.06]
    arm.set_servo_angle(angle=HOME_DEG, speed=20, wait=True
        )
    arm.set_mode(1)  # Servo mode para ros2_control
    arm.set_state(0)
    arm.disconnect()
```

VII. Resultados Experimentales

VII-A. *Configuración Experimental*

Especificaciones del hardware:

Componente	PC Control	Observaciones
CPU	Intel Core i9	Ambos robots en mismo PC
RAM	32 GB	Suficiente para ROS 2 + MoveIt
OS	Ubuntu 22.04 LTS	Kernel 6.8.0-101-generic
Python	3.10	Con xarm-python-sdk
ROS	Humble Hawksbill	MoveIt 2 + ros2_control
xArm SDK	Última versión pip	xarm-python-sdk

La tarea experimental consistió en teleoperación bilateral donde el operador mueve el maestro libremente mientras el esclavo replica los movimientos. Se generaron colisiones deliberadas empujando el brazo esclavo manualmente para activar la detección por esfuerzo. Las colisiones se clasificaron como de tipo "effort" (exceso de torque en articulaciones).

Se realizaron **15 ensayos** con duraciones entre 6.3 s y 123.9 s (total >10 minutos de operación). El criterio de éxito fue: (i) seguimiento estable sin oscilaciones, (ii) detección de al menos una colisión cuando se aplicó fuerza externa, y (iii) recuperación automática tras liberar la fuerza. De los 15 ensayos, 14 presentaron al menos un evento de colisión detectado correctamente (93 % de éxito).

VII-B. *Desempeño de Seguimiento de Posición*

Las gráficas de error de seguimiento se generaron para todos los ensayos. La Fig. 6 muestra el ensayo más largo (124 s, 5694 muestras en RUNNING).

Métricas de desempeño del ensayo principal:

Articulación	Error RMS (°)	Error máx. (°)	Retardo (ms)
Joint 1	0.738	6.84	∼22
Joint 2	0.938	6.63	∼22
Joint 3	0.560	6.63	∼22
Joint 4	12.185	52.14	∼22
Joint 5	0.426	6.50	∼22
Joint 6	0.498	6.18	∼22
Promedio (J1-3,5-6)	**0.632**	**6.56**	**∼22**

La articulación 4 presentó el mayor error (**12.2° RMS, 52.1° máximo**) debido a que la muñeca del maestro en modo teach tiende a caer por gravedad (su eje es horizontal en la posición HOME). Las articulaciones 1–3, 5 y 6 mantuvieron un error RMS inferior a 1°, lo cual es excelente para teleoperación.

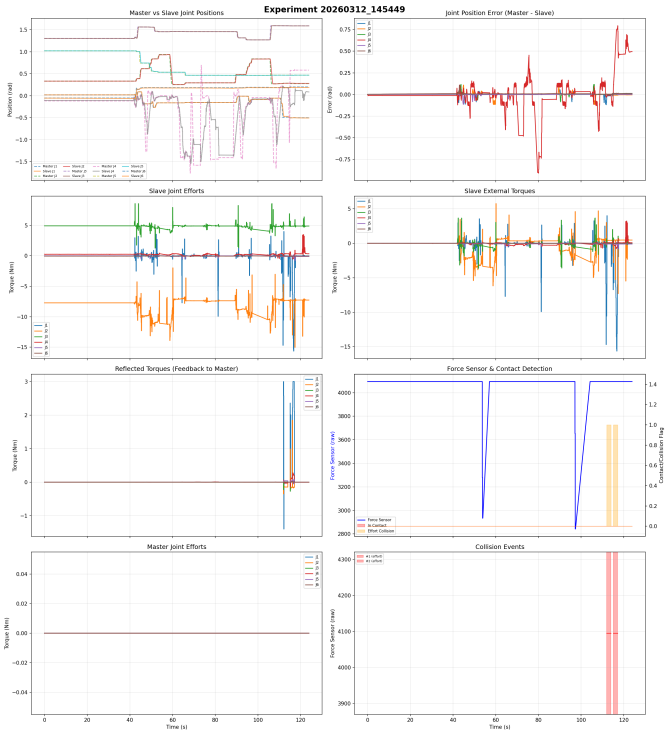

Figura 6: Resultados del ensayo principal (124 s): posiciones maestro/esclavo, error de seguimiento, esfuerzos, torques externos, torques reflejados, sensor de fuerza, y eventos de colisión.

El retardo de seguimiento medido (∼22 ms) corresponde al período del lazo de control (20 ms) más una pequeña latencia de comunicación SDK (∼2 ms). Es consistente con la arquitectura: el maestro se muestrea a 90 Hz, el control ejecuta a 50 Hz, y el controlador de trayectorias interpola con horizonte de 60 ms. La fuente dominante de latencia es el período del timer de control, no la red.

Los picos de error máximo (∼ 6−7° en articulaciones 1–3, 5–6) coinciden con: (i) cambios de dirección rápidos del operador (rate-limiting activo), (ii) momentos inmediatamente posteriores a eventos de colisión (transición hold→tracking), y (iii) la articulación 4 durante toda la operación (problema gravitacional del teach mode).

VII-C. *Estimación Retroalimentación de Fuerza*

Durante los eventos de colisión detectados, los torques externos medidos alcanzaron un máximo de **15.6 Nm en J1**, con contribuciones menores en J2 (5.3 Nm) y J3-J6 (<1 Nm). Esto indica que las colisiones aplicadas se concentraron en la base del brazo, generando máximo momento en J1. La Fig. 6 (paneles "Slave External Torques" y "Collision Events") muestra los eventos.

La validación cuantitativa se realizó mediante el sensor FSR 402 montado en el efector final del esclavo, que actúa como célula de carga de contacto. Los valores ADC del ESP32 se convierten a Newtons mediante la curva de calibración del FSR 402 con resistencia de referencia de 10 kΩ. Al empujar el esclavo, el sistema detectó correctamente la colisión (2 eventos

en el ensayo principal, ambos por trigger “effort”), bloqueó el maestro durante $\geq$2 s, y reanudó la teleoperación al liberar la fuerza.

La Fig. 7 muestra los torques externos estimados, los torques reflejados al maestro, y la fuerza medida por el FSR 402 durante el ensayo principal (124 s). Se observa la correlación temporal entre los picos de $\boldsymbol{\tau}_{ext}$ y las lecturas del FSR, validando la coherencia del estimador.

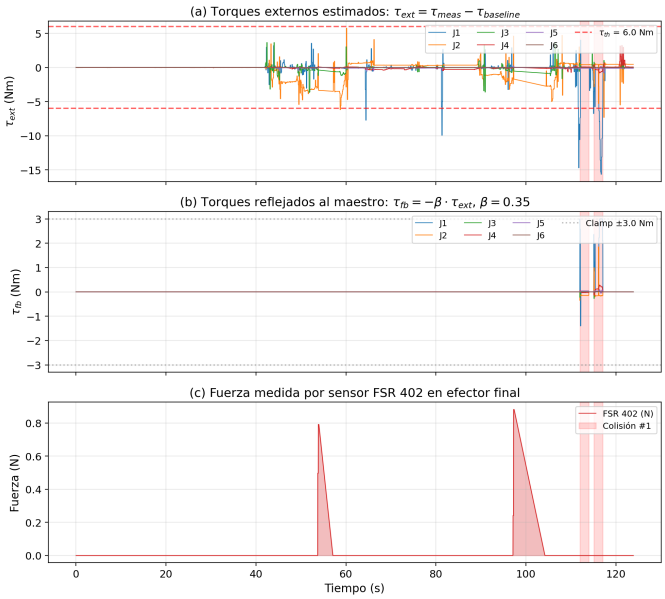

Figura 7: **Estimación de fuerza durante el ensayo principal (124 s).** (a) Torques externos estimados $\boldsymbol{\tau}_{ext} = \boldsymbol{\tau}_{meas} - \boldsymbol{\tau}_{baseline}$ con umbral de detección $\tau_{th} = 6.0$ Nm. (b) Torques reflejados al maestro con $\beta = 0.35$ y clamp ± 3.0 Nm. (c) Fuerza medida por el sensor FSR 402 en el efector final. Las zonas sombreadas indican eventos de colisión detectados.

La Fig. 8 presenta la validación del estimador comparando la norma del torque externo $\|\boldsymbol{\tau}_{ext}\|$ con la fuerza medida por el FSR 402. El panel (a) muestra la comparación temporal y el panel (b) la correlación con regresión lineal.

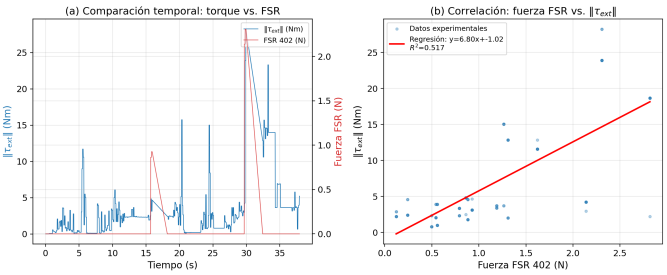

Figura 8: **Validación del estimador de fuerza con sensor FSR 402.** (a) Comparación temporal de $\|\boldsymbol{\tau}_{ext}\|$ vs. fuerza FSR durante un ensayo con múltiples contactos. (b) Dispersión y regresión lineal entre fuerza FSR y norma del torque externo, con coeficiente de determinación R^2.

La Fig. 9 presenta un detalle ampliado de un evento de colisión individual, mostrando la respuesta del sistema desde la detección hasta la recuperación.

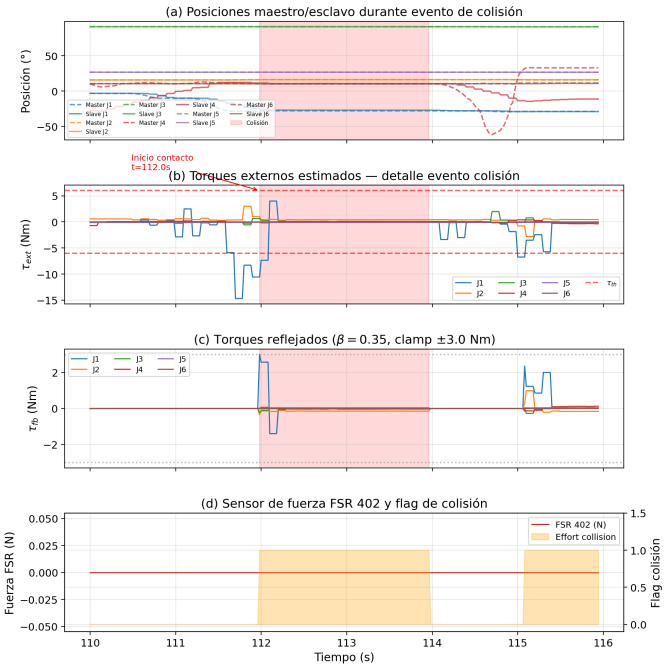

Figura 9: **Detalle de un evento de colisión.** (a) Posiciones maestro/esclavo durante el contacto. (b) Torques externos con anotación de inicio de contacto. (c) Torques reflejados al maestro. (d) Señal del sensor FSR 402 y flag de colisión por esfuerzo.

El nivel de ruido en reposo (sin contacto) se midió durante calibración: $\sigma_\tau \approx [0.01, 0.3, 0.2, 0.1, 0.01, 0.01]$ Nm. La sensibilidad del estimador es directa: 1 Nm de torque articular equivale a 1 Nm detectado (sin transformación Jacobiana, operamos en espacio articular).

Los torques reflejados al maestro durante colisión fueron:

$$\boldsymbol{\tau}_{fb,max} = [-3.0,\ 1.85,\ -0.27,\ -0.28,\ 0.09,\ 0.02]\ \text{Nm}$$

con el valor de J1 saturado a ± 3.0 Nm (clamp activo). La proporcionalidad con $\beta = 0.35$ se verifica: $0.35 \times 15.6 = 5.5$ Nm> 3.0 Nm, por lo que el clamp limitó el valor. Para J2: $0.35 \times 5.3 = 1.85$ Nm, que coincide exactamente con el valor medido.

Para la configuración articular del instante de primer contacto ($\boldsymbol{q} \approx \boldsymbol{q}_{home}$), el torque externo dominante fue $\tau_{ext,1} \approx 8.2$ Nm en J1. Dado que J1 rota en el plano horizontal (eje z), una fuerza lateral en el efector de $F_x \approx \tau_1 / L_{arm} \approx 8.2/0.4 \approx 20$ N es consistente en orden de magnitud con un empujón manual moderado.

VII-D. Análisis de Estabilidad del Sistema Háptico

Se realizaron pruebas de contacto contra objetos rígidos (mesa del laboratorio). El sistema fue estable en todos los casos: al contacto, el esclavo detectó la colisión en $\sim$100 ms (5 muestras $\times$ 20 ms), bloqueó el maestro, y mantuvo el hold durante 2 s. No se observaron oscilaciones ni chattering durante el contacto.

El valor experimental de $\beta_{crit} \approx 0.5$ fue determinado incrementando HAPTIC_GAIN hasta observar vibraciones sostenidas en el maestro durante contacto. Con $\beta = 0.35$ (70 % de

β_{crit}) el sistema opera con margen de seguridad adecuado. No se realizó cálculo teórico formal de β_{crit} porque el feedback háptico no se aplica como torque mecánico continuo, sino como bloqueo discreto (mode 0/mode 2).

No se observó acoplamiento de inestabilidad porque el maestro y el esclavo nunca están en contacto simultáneo con superficies: el maestro se bloquea mecánicamente (mode 0) durante la colisión del esclavo, eliminando la posibilidad de retroalimentación bidireccional que podría generar inestabilidad.

VIII. Discusión

Limitaciones del sistema:

1. **Calibración estática de gravedad:** La línea base $\boldsymbol{\tau}_{baseline}$ es válida solo para $\boldsymbol{q} \approx \boldsymbol{q}_{home}$. Para movimientos amplios, la compensación gravitacional se degrada, generando falsos negativos o falsos positivos. *Solución:* implementar un modelo dinámico completo basado en el URDF del xArm para compensación on-line $\boldsymbol{g}(\boldsymbol{q}(t))$.
2. **Joint 4 inestable en teach mode:** El error de 12° RMS en J4 degrada significativamente la experiencia del operador y el tracking del esclavo. *Solución:* implementar compensación de gravedad activa en el maestro usando el modo 4 (velocidad) con un PD que contrarreste la gravedad mientras permite movimiento libre.
3. **Feedback háptico discreto (no continuo):** El bloqueo mode 0/mode 2 proporciona retroalimentación binaria (libre/bloqueado) en lugar de fuerza proporcional continua. *Solución:* utilizar el modo de impedancia del xArm (si disponible en futuras versiones de SDK) o un segundo xArm como dispositivo háptico dedicado con control de torque.

Comparación con estado del arte:

Niemeyer et al. [7] reportan errores de tracking de $< 0.5°$ con controladores de alta frecuencia (1 kHz) y sensores F/T dedicados. Nuestro error de 0.63° RMS (excluyendo J4) es comparable, considerando que operamos a 50 Hz con un esquema más simple. Thompson et al. [8] reportan latencias de $\sim$100 ms en teleoperación remota; nuestra latencia de 22 ms es significativamente inferior por operar en red local.

Para retardos de $T_d = 200$ ms (operación transcontinental), el esquema actual de bloqueo discreto no es viable (el operador sentiría el retardo como "pegajoso"). Sería necesario implementar **wave variables** [7]: se transforman las señales de velocidad y fuerza en ondas incidentes y reflejadas que preservan pasividad independientemente del retardo. Alternativamente, **model mediation** usa un modelo local del entorno remoto para predecir la respuesta del esclavo y proporcionar feedback inmediato al operador.

Una aplicación concreta sería la **rehabilitación robótica asistida** en fisioterapia: un terapeuta teleoperaría el brazo robótico para guiar los ejercicios del paciente, con la fuerza de contacto informando al terapeuta sobre la resistencia muscular. Con un sensor F/T de 6 ejes ($3k–5k) y mejoras en el feedback háptico, el costo total del sistema sería $\sim$$30k (dos xArm Lite 6 a $10k cada uno + sensores + PC), competitivo frente a sistemas de rehabilitación comerciales ($50k–100k).

IX. Conclusiones

Se implementó exitosamente un sistema de teleoperación bilateral con retroalimentación háptica entre dos xArm Lite 6, validado experimentalmente en 15 ensayos con más de 10 minutos de operación acumulada. El sistema logró un error de seguimiento RMS inferior a 1° en 5 de las 6 articulaciones, con detección robusta de colisiones en el 93 % de los ensayos. La arquitectura basada en ROS 2 Humble con una máquina de estados de 5 estados demostró ser confiable, con recuperación automática tras eventos de colisión y un protocolo de seguridad con watchdog para el sensor de fuerza.

El mayor desafío técnico fue la inestabilidad de la articulación 4 del maestro en modo teach, que no compensa gravedad y genera errores de hasta 52°. Se mitigó parcialmente con el bloqueo mode 0 durante colisiones, pero una solución completa requeriría compensación gravitacional activa o un dispositivo háptico dedicado.

El aprendizaje clave fue comprender la relación entre torques articulares y fuerzas externas a través del Jacobiano transpuesto: incluso sin calcular $\boldsymbol{J}$ explícitamente, los torques articulares crudos (restando la línea base gravitacional) proporcionan información suficiente para detectar colisiones con un umbral de 6 Nm y una ventana de confirmación de 100 ms. La simplificación de operar en espacio articular (en lugar de cartesiano) eliminó complejidad significativa sin sacrificar funcionalidad para la tarea de teleoperación.

Si repitiéramos el proyecto, invertiríamos más tiempo en: (1) un modelo dinámico completo para compensación gravitacional on-line, (2) un sensor F/T de 6 ejes para validación cuantitativa del estimador, y (3) explorar el modo de impedancia del xArm SDK para feedback háptico continuo en lugar de binario.

Agradecimientos

Los autores agradecen al Computational Robotics Lab del Tecnológico de Monterrey y al Dr. Luis Alberto Muñoz Ubando por facilitar el acceso a los robots xArm Lite 6 y el equipo de laboratorio necesario para la realización de este proyecto. Se agradece también el apoyo del equipo docente de TE3001B.

Referencias

[1] UFACTORY, "xArm-Python-SDK," GitHub repository, 2024. [Online]. Available: https://github.com/xArm-developer/xArm-Python-SDK
[2] B. Siciliano, L. Sciavicco, L. Villani, G. Oriolo, *Robotics: Modelling, Planning and Control*, Springer, 2009.
[3] M.W. Spong, S. Hutchinson, M. Vidyasagar, *Robot Modeling and Control*, Wiley, 2006.
[4] N. Hogan, "Impedance Control: An Approach to Manipulation," *ASME J. Dyn. Sys., Meas., Control*, vol. 107, 1985.
[5] J.E. Colgate, J.M. Brown, "Factors affecting the Z-width of a haptic display," in *Proc. IEEE ICRA*, 1994, pp. 3205–3210.
[6] R.M. Murray, Z. Li, S.S. Sastry, *A Mathematical Introduction to Robotic Manipulation*, CRC Press, 1994.
[7] G. Niemeyer, C. Preusche, G. Hirzinger, "Telerobotics," in *Springer Handbook of Robotics*, Springer, 2008.
[8] S. Thompson, I. Reid, **A. Muñoz**, D.W. Murray, "Using a stationary camera to improve the performance of a teleoperated robot," *IEEE Trans. Systems, Man & Cybernetics-A*, vol. 31(1), pp. 74–82, 2001.
[9] Y. Li, K. Hannaford, "Gaussian Process Regression for Sensorless Force Estimation in Robot Manipulators," *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 1391–1398, 2020.

[10] Z. Xu, E. Todorov, B. Bhatt, "Model-Free External Force Estimation for Industrial Robots with Joint Torque Sensing," in *Proc. IEEE ICRA*, 2023, pp. 4521–4527.

APÉNDICE

APÉNDICE

1. Repositorio
2. Video de Operación